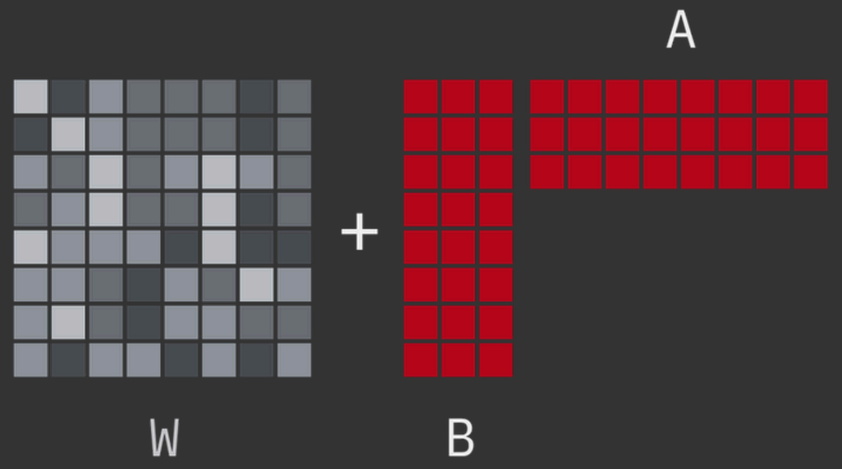
Clase 7
Ajuste fino eficiente

[CC66Q] Procesamiento de Lenguaje Natural

Andrés Abeliuk y Fabrizzio Pezzolla

Diplomado en Inteligencia Artificial





Clase de hoy

Parte 1 — **Ajuste fino supervisado (SFT)**
Qué se entrena y cuántos datos

Parte 2 — **Cuantización**
Formatos numéricos, absmax, métodos y BitNet

Parte 3 — **Ajuste fino eficiente**
LoRA y QLoRA

Notebook — Cargar el modelo, el antes, los datos y el entrenamiento

Break — Mientras entrena

Notebook — El después y el ejercicio propuesto

Modelo base

lo que le damos

```
### Instrucción:
Escribe un correo breve para pedir una reunión el lunes.

### Respuesta:
```

¿Qué va a escribir?

Modelo base

lo que le damos

```
### Instrucción:
Escribe un correo breve para pedir una reunión el lunes.

### Respuesta:
```

lo que escribe

```
Hola, me gustaría reunirme con usted el lunes a las 10:00.
### Instrucción:
Escribe un correo breve para pedir una reunión el martes.
### Respuesta:
Hola, me gustaría reunirme con usted el martes a las 10:00.
### Instrucción:
Escribe un correo breve para pedir una reunión el miércoles.
…
```

Parte 1

Ajuste fino supervisado (SFT)

Qué se entrena y cuántos datos

Ajuste fino supervisado (SFT)

Qué se entrena

- SFT usa la misma pérdida de la clase 5: predecir el token siguiente.
- La pérdida cuenta solo los tokens de la respuesta. La instrucción entra solo como contexto.
- El modelo también aprende a producir el token de fin, y así sabe cuándo parar.



Ajuste fino supervisado (SFT)

Cuántos datos

- Alpaca (2023) ajustó LLaMA de 7.000 millones de parámetros con 52.000 ejemplos generados por un modelo de OpenAI.
- LIMA (2023) ajustó LLaMA de 65.000 millones de parámetros con 1.000 ejemplos elegidos por su calidad y variedad.
- LIMA superó al mismo LLaMA de 65.000 millones ajustado con los 52.000 ejemplos de Alpaca.
- Contra GPT-4, las respuestas de LIMA empataron o fueron preferidas en el 43 % de los casos.

Taori et al. (2023); Zhou et al. (2023)

Ajuste fino supervisado (SFT)

Alineamiento superficial

- La hipótesis de LIMA: casi todo lo que el modelo sabe lo aprende en el preentrenamiento.
- El ajuste fino solo le enseña el formato: cómo responder, en qué tono y cuándo parar.
- En los experimentos de LIMA, más ejemplos no mejoraron las respuestas, pero más calidad y variedad sí.

Zhou et al. (2023)

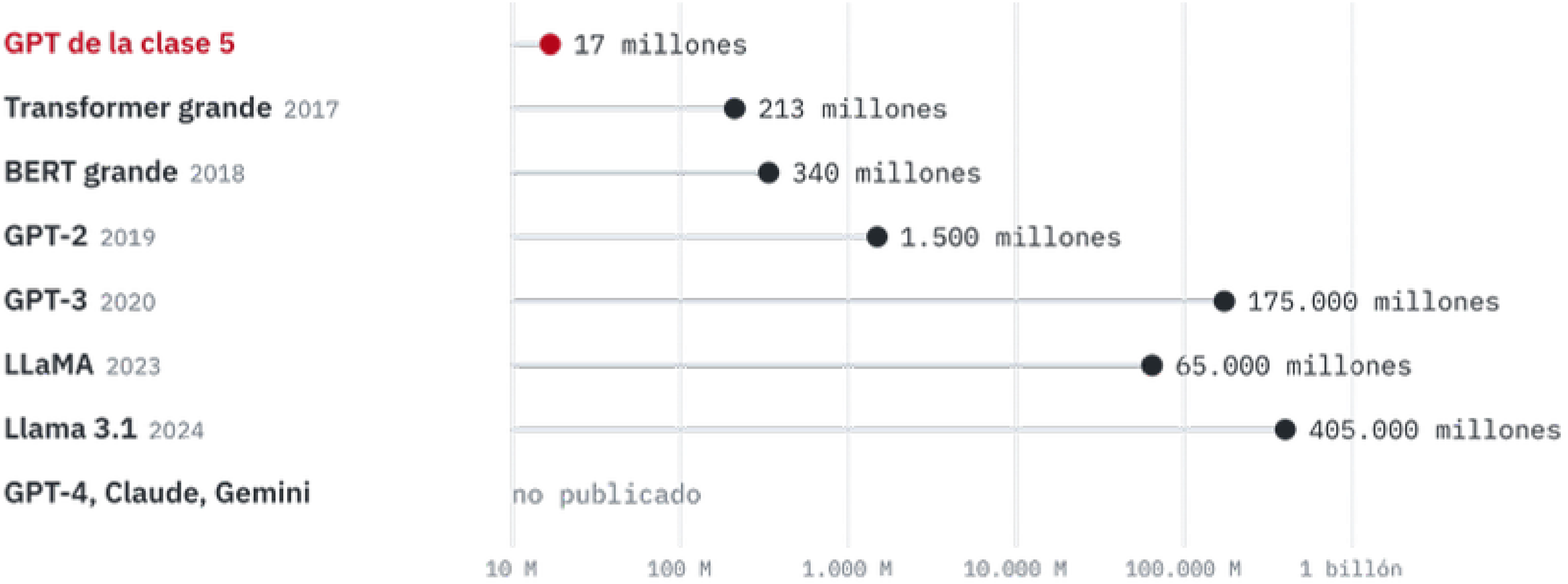
Parte 2

Cuantización

Formatos numéricos, absmax, métodos y BitNet

Tamaño de los modelos

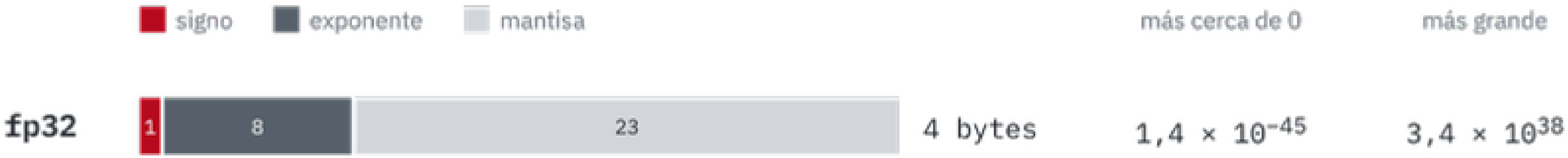
Parámetros



Vaswani et al. (2017); Devlin et al. (2018); Radford et al. (2019); Brown et al. (2020); Touvron et al. (2023); Meta (2024)

Formatos numéricos

fp32



Hasta 2017, las redes se entrenaban en fp32: 4 bytes por número.

Formatos numéricos

fp16

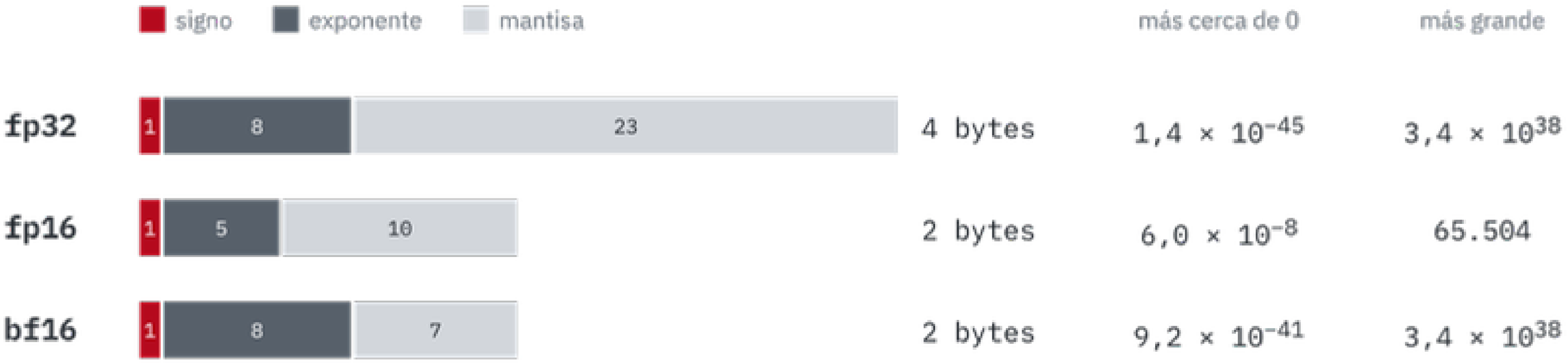


fp16 ocupa la mitad de memoria, pero tiene menos rango: sobre 65.504, un número se vuelve infinito.

Micikevicius et al. (2017)

Formatos numéricos

bf16



Google Brain creó bf16 para sus TPU: el rango de fp32 en 2 bytes, con menos precisión.

Memoria de los pesos = parámetros × bytes por número

Wang y Kanwar (2019)

Inferencia

Cuánto cuesta generar

- Los pesos de GPT-3 en 16 bits ocupan 350 GB.
- Generar un token cuesta unas 2 operaciones por parámetro.
- Para cada token, la GPU lee todos los pesos de su memoria y hace apenas dos operaciones con cada uno.
- Por eso, al generar, el límite es la velocidad con que la GPU lee su memoria.

Kaplan et al. (2020); Hu et al. (2021); Frantar et al. (2022)

Cuantización

Compresión con pérdida

- Cuantizar es guardar cada número con menos bits, redondeándolo al valor más cercano de una lista corta.
- Es compresión con pérdida, como reducir los colores de una foto.
- En 2011, Google reconoció voz en tiempo real en CPU, con pesos y activaciones en 8 bits.
- En 2017, Google hizo redes de visión que calculan solo con enteros, para correr en teléfonos.



millones de colores



8 colores

Vanhoucke et al. (2011); Jacob et al. (2017)

Cuantización

Absmax

pesos	0,12	−0,53	0,88	−0,07	0,31	−0,40
× 7 / 0,88	0,95	−4,22	7,00	−0,56	2,47	−3,18
redondeo	**1**	**−4**	**7**	**−1**	**2**	**−3**
× 0,88 / 7	0,126	−0,503	0,880	−0,126	0,251	−0,377

$$q = \text{round}\left(\frac{7}{\max|w|}\, w\right)$$

El error de cada capa entra en la siguiente, así que los errores se acumulan a lo largo del modelo.

Cuantización

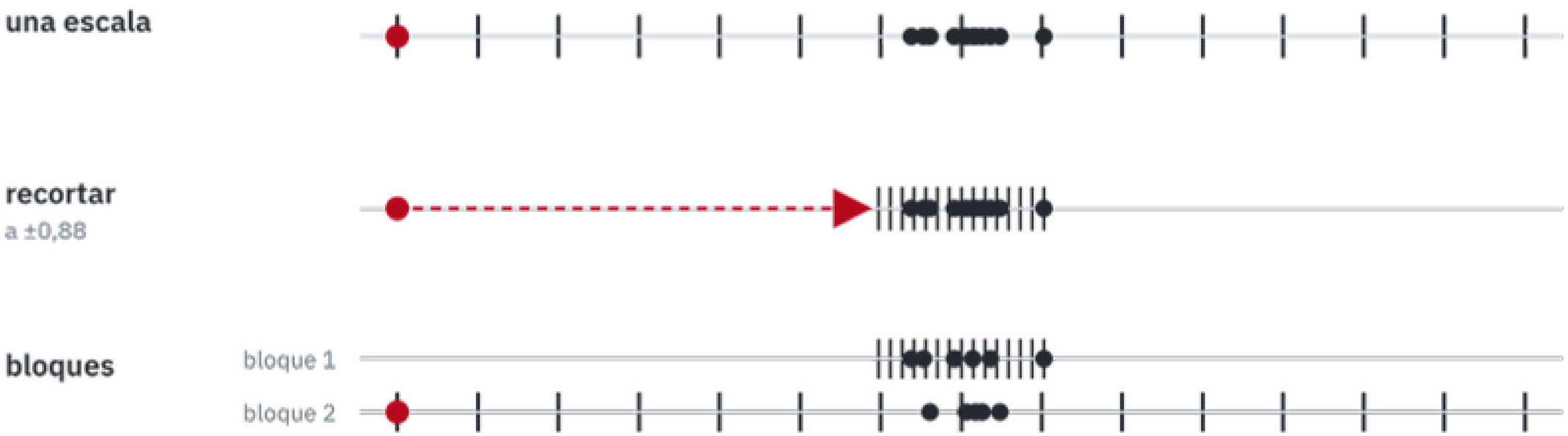
Un valor extremo

una escala



Cuantización

Recortar o dividir en bloques

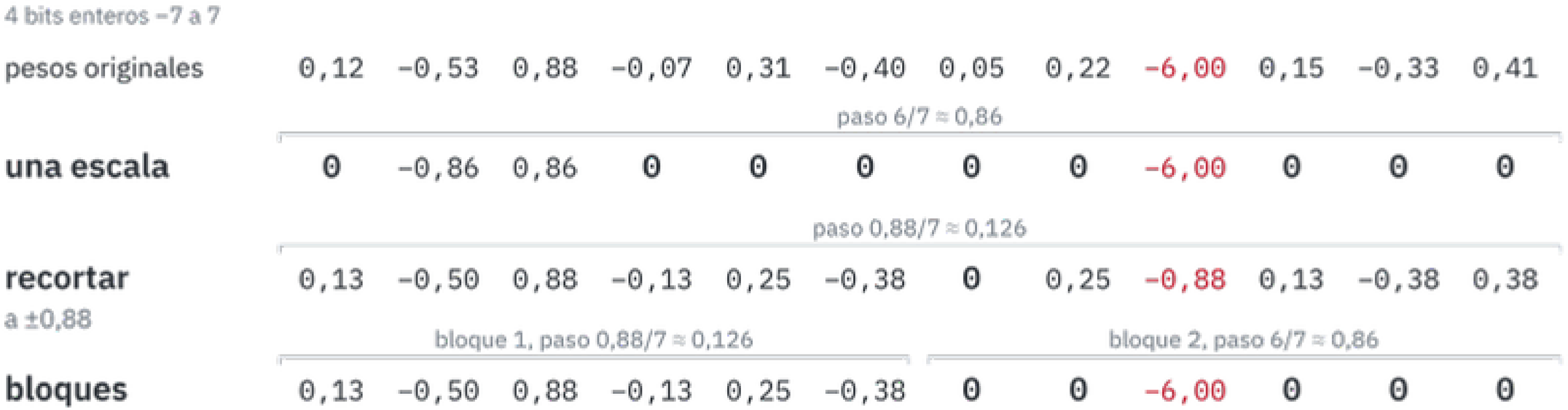


Puntos: pesos. Marcas: valores que se pueden guardar.

Dettmers et al. (2023)

Cuantización

Recortar o dividir en bloques



Arriba, los doce pesos originales. Debajo, el valor que recupera cada método.

Dettmers et al. (2023)

Métodos de cuantización

Lo que tienen en común

- Todos usan una escala por bloque o por fila y redondean cada peso a una grilla.
- Todos se aplican a un modelo ya entrenado, porque volver a entrenarlo sería demasiado caro.
- Todos chocan con el mismo problema: unos pocos valores muy grandes, que a veces sí importan.
- Cada método trata esos valores grandes de una manera distinta.

Métodos de cuantización

LLM.int8(), GPTQ y AWQ



- LLM.int8() guarda los pesos en 8 bits y convierte también las activaciones a 8 bits, para multiplicar con enteros.
- Cada fila de las activaciones y cada columna de los pesos lleva su propia escala absmax.
- Unas pocas dimensiones de las activaciones, cerca del 0,1 %, tienen valores enormes. Esas se multiplican aparte, en 16 bits.
- El resto, más del 99,9 % de los valores, se multiplica en 8 bits, y al final se suman los dos resultados.

Dettmers et al. (2022)

Métodos de cuantización

LLM.int8(), GPTQ y AWQ



- GPTQ cuantiza solo los pesos, a 4 o 3 bits, y deja las activaciones en 16.
- Recorre cada matriz de pesos columna por columna, redondeando una columna a la vez.
- Después de redondear una columna, ajusta las que faltan para cancelar el error que se introdujo.
- Mide ese error con 128 textos de ejemplo: busca que la salida de la capa cambie lo menos posible.

Dettmers et al. (2022); Frantar et al. (2022)

Métodos de cuantización

LLM.int8(), GPTQ y AWQ

LLM.int8()	GPTQ	AWQ
2022	2022	2023
8 bits	4 bits	4 bits

- AWQ, igual que GPTQ, cuantiza solo los pesos, a 4 bits, y deja las activaciones en 16.
- No todos los pesos importan igual: importan más los que se multiplican por activaciones grandes.
- Antes de redondear, AWQ multiplica esos pesos por un factor, para que pierdan menos precisión.
- Para que el resultado no cambie, la activación que les llega se divide por el mismo factor.

Dettmers et al. (2022); Frantar et al. (2022); Lin et al. (2023)

Métodos de cuantización

Cuánto se pierde

perplejidad	Llama-2 7B	13B	70B
fp16	5,47	4,88	3,32
redondeo simple	5,73	4,98	3,46
GPTQ	5,69	4,98	3,42
AWQ	**5,60**	**4,97**	**3,41**

Perplejidad de Llama-2 en WikiText-2, con los pesos en 4 bits y bloques de 128. Menos es mejor.

Con los pesos en 4 bits, el modelo ocupa cerca de la cuarta parte de la memoria que en 16 bits.

Lin et al. (2023), tabla 4

BitNet b1.58

Un bit y medio

- En las capas lineales, cada peso vale −1, 0 o +1: $\log_2 3 \approx 1{,}58$ bits.
- Multiplicar por un peso así es sumar, restar o saltarse el valor.
- Se entrena con estos pesos desde el principio, en vez de cuantizar un modelo ya entrenado.
- Desde 3.000 millones de parámetros, iguala en perplejidad a un modelo igual entrenado en 16 bits.

Ma et al. (2024)

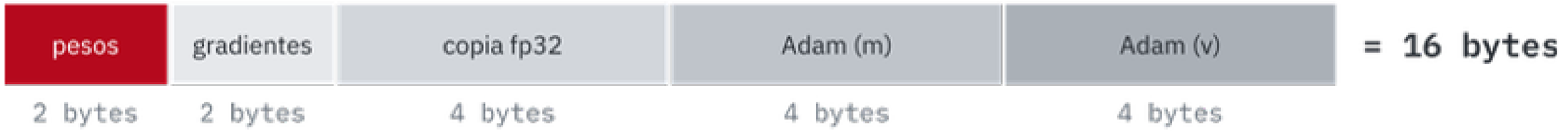
Parte 3

Ajuste fino eficiente

LoRA y QLoRA

Ajuste completo

Cuánta memoria



- Entrenar con Adam en precisión mixta ocupa unos 16 bytes por parámetro: pesos, gradientes, copia en fp32 y Adam.
- Entrenar GPT-2, de 1.500 millones de parámetros, necesita al menos 24 GB. Sus pesos solos ocupan 3 GB.
- Cada token de entrenamiento cuesta unas 6 operaciones por parámetro, el triple que generar.

Rajbhandari et al. (2020); Kaplan et al. (2020)

Ajuste completo

Activaciones

- Las activaciones son los resultados intermedios de la pasada hacia adelante, que se guardan para calcular los gradientes.
- En GPT-2, con secuencias de 1.024 tokens y batch de 32, las activaciones suman unos 60 GB más.
- Gradient checkpointing guarda solo algunas activaciones y recalcula las demás cuando se necesitan.
- Megatron (2022) recalcula solo las de la atención (scores, softmax y dropout), que ocupan mucha memoria y cuestan poco de calcular.

Rajbhandari et al. (2020); Chen et al. (2016); Korthikanti et al. (2022)

Ajuste completo

Una copia por tarea

- GPT-3 (2020) tiene 175.000 millones de parámetros.
- Cada tarea ajustada produce un modelo completo nuevo, de 350 GB.
- Según el paper de LoRA, ajustarlo entero con Adam usa 1,2 TB de memoria de GPU.
- ¿Hace falta cambiar todos los pesos?

Hu et al. (2021)

Dimensión intrínseca

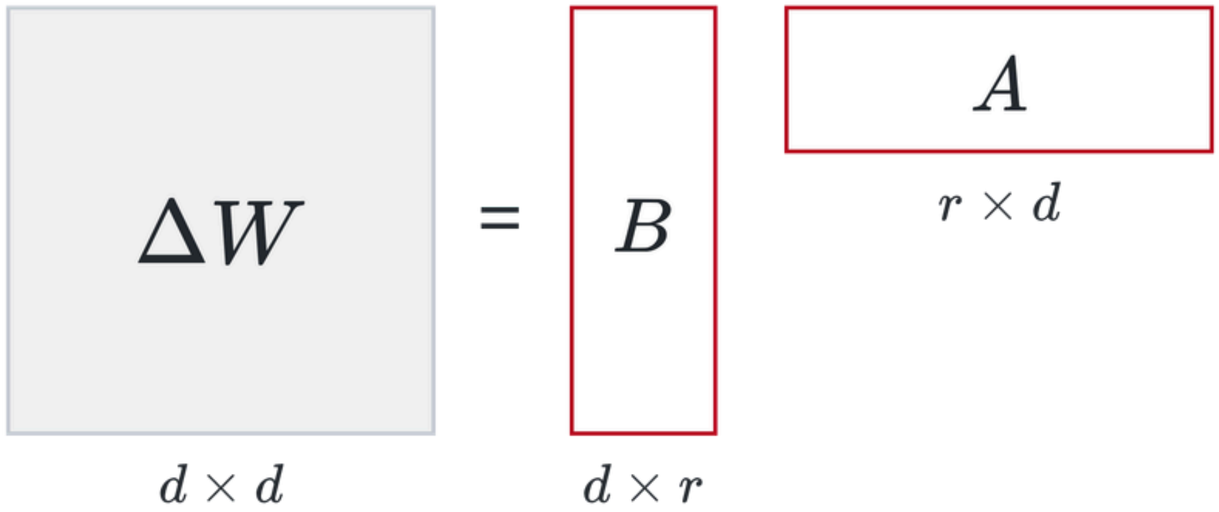
Pocas direcciones bastan

- ¿En cuántas direcciones tienen que moverse los parámetros durante un entrenamiento?
- En 2018 entrenaron redes moviendo los parámetros solo en unas pocas direcciones elegidas al azar.
- La dimensión intrínseca es la menor cantidad de direcciones que alcanza el 90 % del rendimiento del entrenamiento normal.
- En una tarea, RoBERTa Large llegó a ese 90 % entrenando unos 200 números.

Li et al. (2018); Aghajanyan et al. (2020)

LoRA

Dos matrices delgadas



- LoRA escribe el cambio de los pesos, ΔW, como el producto de dos matrices delgadas, B y A, de ancho r.
- En vez de los d × d números de ΔW, se entrenan 2 × d × r, muchos menos cuando r es chico.

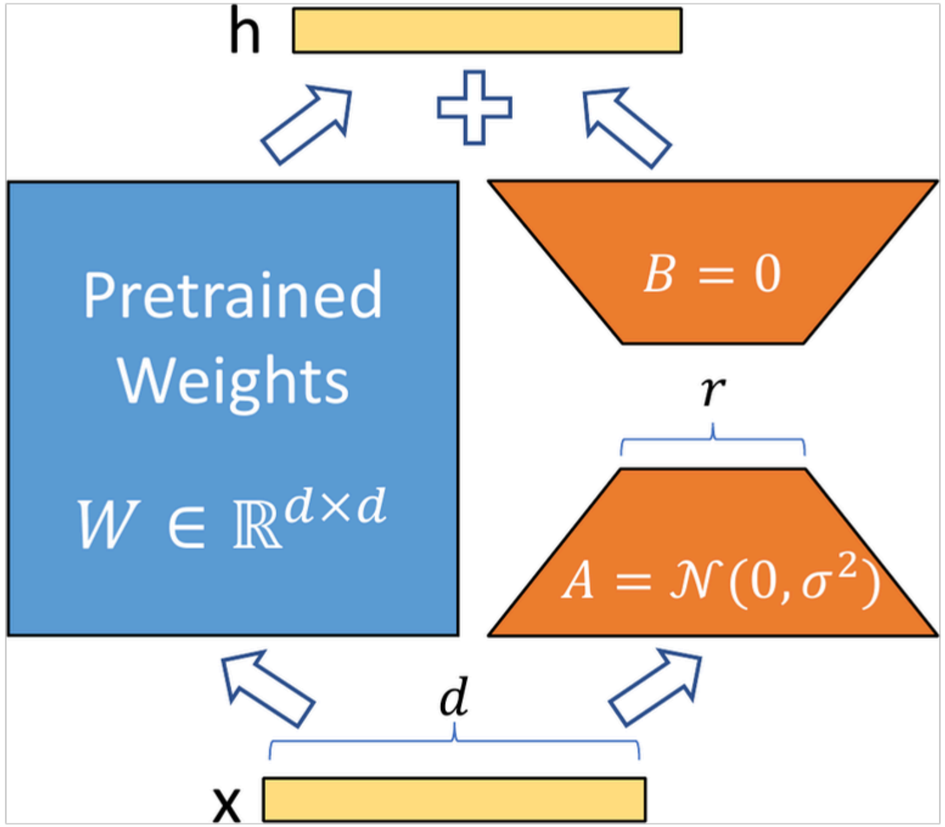
Hu et al. (2021)

LoRA

Cómo se entrena

- Los pesos preentrenados W quedan congelados: no se actualizan.
- Se entrena solo la corrección BA, cuya salida se suma a la de W.
- B empieza en cero y A al azar: al comienzo BA es cero y el modelo es el original.

$$h = Wx + BAx$$



Hu et al. (2021), fig. 1

LoRA

r y α

- Antes de sumar la corrección BA, su salida se multiplica por α/r.
- Dividir por r reduce la necesidad de reajustar los demás hiperparámetros cuando se cambia r.

$$h = Wx + \frac{\alpha}{r} BAx$$

Hu et al. (2021)

LoRA

Qué se gana

- En GPT-3, LoRA con r = 4 en queries y values entrena unos 18 millones de parámetros, 10.000 veces menos.
- La memoria para entrenar baja de 1,2 TB a 350 GB, porque los pesos congelados no necesitan gradientes ni Adam.
- Cada versión ajustada guarda solo B y A: 35 MB, en vez de una copia de 350 GB.

Hu et al. (2021)

LoRA

Qué matrices y qué r

	r	WikiSQL (%)
W_q	8	70,4
W_v	8	73,0
W_q y W_v	**4**	**73,7**
W_q, W_k, W_v y W_o	2	73,7

WikiSQL (%)	r = 1	2	4	8	64
W_q	68,8	69,6	70,5	70,4	70,0
W_q y W_v	**73,4**	**73,3**	**73,7**	**73,8**	**73,5**

- Con el mismo número de parámetros, rinde más adaptar varias matrices con r chico que una sola con r grande.
- En WikiSQL, con queries y values, r = 1 ya rinde casi igual que r = 64.
- Después, los autores de QLoRA igualaron al ajuste completo poniendo LoRA en todas las capas lineales.

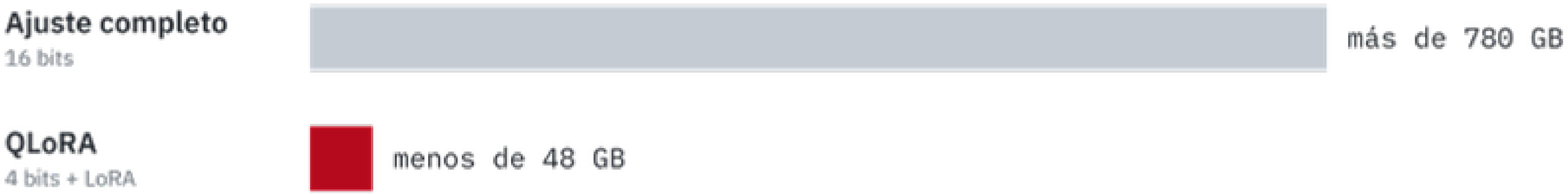
Exactitud en WikiSQL (% de consultas SQL correctas) de GPT-3 con LoRA, con 18 millones de parámetros entrenables. El ajuste completo logra 73,8 %.

Hu et al. (2021), tablas 5 y 6; Dettmers et al. (2023)

QLoRA

La idea

QLoRA guarda el modelo congelado en 4 bits y entrena LoRA encima, en 16 bits.

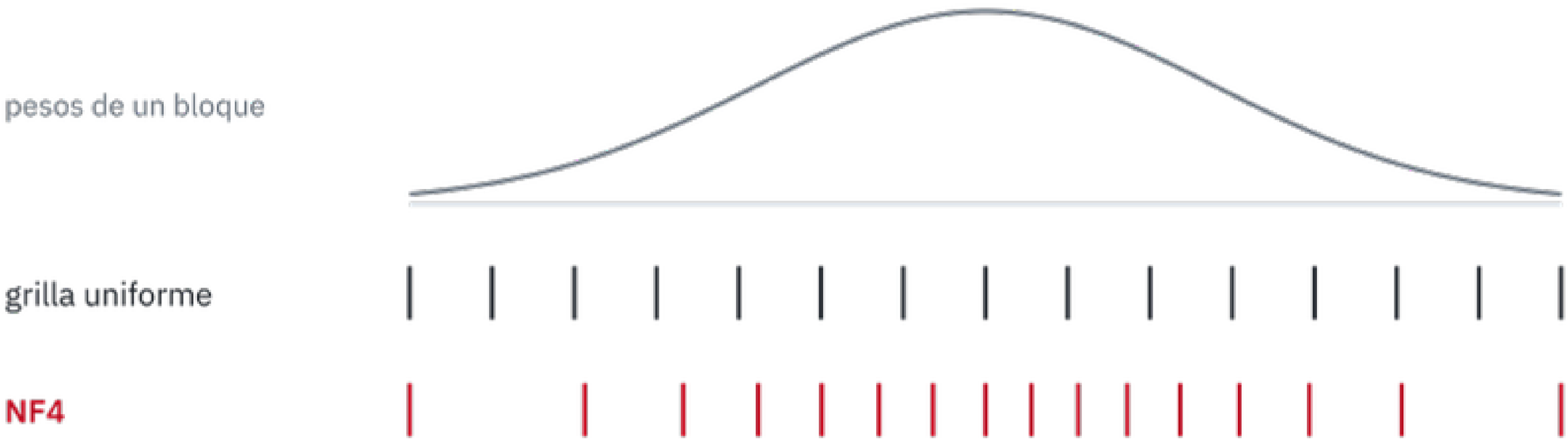


Guanaco, entrenado así durante 24 horas en una GPU, llegó al 99,3 % del puntaje de ChatGPT en el benchmark de Vicuna.

Dettmers et al. (2023)

QLoRA

NF4



NF4 pone sus niveles más juntos cerca de cero, donde se concentran los pesos.

Dettmers et al. (2023)

Mixture of Experts (MoE)

Extra: muchos parámetros, pocos activos

- En un MoE, la feed-forward de cada bloque se reemplaza por varias redes, los expertos.
- Una red chica, el router, elige qué expertos procesan cada token.
- Mixtral (2024) tiene 8 expertos por capa y usa 2 para cada token.
- Cada token usa 13.000 millones de parámetros, pero los 47.000 millones tienen que estar en memoria.

Shazeer et al. (2017); Jiang et al. (2024)

Automejora

Extra: ¿y si el modelo decide el ajuste?

- SEAL (2025): el modelo escribe sus propios datos de ajuste fino, una autoedición.
- Con aprendizaje por refuerzo se premian las autoediciones que dejan mejor al modelo.
- Weight-space programming (2026): un LLM lee los pesos de otro modelo y los edita razonando, sin entrenar.
- Editando el 0,016 % de los pesos de Qwen3.5-0.8B, su puntaje en MATH-500 subió de 7,9 % a 48,1 %.

Zweiger et al. (2025); Takehi et al. (2026)

Referencias

Taori et al. (2023), «Alpaca: A Strong, Replicable Instruction-Following Model»

Zhou et al. (2023), «LIMA: Less Is More for Alignment»

Kaplan et al. (2020), «Scaling Laws for Neural Language Models»

Micikevicius et al. (2017), «Mixed Precision Training»

Wang y Kanwar (2019), «BFloat16: The secret to high performance on Cloud TPUs»

Vanhoucke et al. (2011), «Improving the speed of neural networks on CPUs»

Jacob et al. (2017), «Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference»

Dettmers et al. (2022), «LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale»

Frantar et al. (2022), «GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers»

Lin et al. (2023), «AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration»

Shazeer et al. (2017), «Outrageously Large Neural Networks»

Jiang et al. (2024), «Mixtral of Experts»

Ma et al. (2024), «The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits»

Rajbhandari et al. (2020), «ZeRO: Memory Optimizations Toward Training Trillion Parameter Models»

Chen et al. (2016), «Training Deep Nets with Sublinear Memory Cost»

Korthikanti et al. (2022), «Reducing Activation Recomputation in Large Transformer Models»

Li et al. (2018), «Measuring the Intrinsic Dimension of Objective Landscapes»

Aghajanyan et al. (2020), «Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning»

Hu et al. (2021), «LoRA: Low-Rank Adaptation of Large Language Models»

Dettmers et al. (2023), «QLoRA: Efficient Finetuning of Quantized LLMs»

Jurafsky y Martin, Speech and Language Processing, 3.ª ed., capítulo 8

BERTIN GPT-J-6B

Zweiger et al. (2025), «Self-Adapting Language Models»

Takehi et al. (2026), «Sparks of Weight-Space Programming»